

- using

$$mgh$$

, all of the gravitational potential energy of

$$h_1$$

will be transferred into the starting velocity of the point at

$$h_2$$

- so we get

$$mgh_1 = \frac{1}{2}mv^2$$

- so final velocity

$$v = \sqrt{\frac{2mgh_1}{m}}$$

– for the marble falling down

$$h_2$$

, we can derive time taken

*

$$s = ut + \frac{1}{2}at^2$$

*

$$h_2 = 0 \times t + \frac{1}{2}(g)t^2$$

*

$$t = \sqrt{\frac{2h_2}{g}}$$

- model 1

–

$$s = ut + \frac{1}{2}at^2$$

–

$$s = \sqrt{\frac{2mgh_1}{m}} * \sqrt{\frac{2h_2}{g}} + 0 \times \dots$$

–

$$s = 2\sqrt{h_1h_2}$$

– (

$$s$$

here being

$$d$$

)

- for model [[2]], we have to include angular velocity

—

$$mgh_1 = \frac{1}{2}mv^2 + \frac{1}{2}I\omega^2$$

—

$$mgh_1 = \frac{1}{2}mv^2 + \frac{1}{2}\left(\frac{2}{5}mr^2\right)\frac{v^2}{r^2}$$

—

$$v = \sqrt{\frac{10}{7}gh_1}$$

- model [[2]]

—

$$s = \sqrt{\frac{10}{7}gh_1} \times \sqrt{\frac{2h_2}{g}} + 0 \times \dots$$

—

$$s = 2\sqrt{\frac{5}{7}h_1h_2}$$

— (

s

here being

d

)

- visual [[test]]

— we visually measured a

$$\mu \approx 85.7cm, \sigma \approx 1.5cm \implies SE = 0.1$$

for black marks

— we visually measured a

$$\mu \approx 85.2cm, \sigma \approx 1.4cm \implies SE = 0.1$$

for red marks

- this is the visual measurement for both, as they both are so similar that the call was made that we should just [[do]] the same for both.
- We did this by choosing the range of the visual 2/3 of the points, and then halving that to get one standard deviation. The mean was also done visually, just the intuitive middle point.

—

- sum actual calculations

— I did all the calculations in python, the following code will show it

—

```
import statistics
from scipy import stats
from uncertainties import ufloat
```

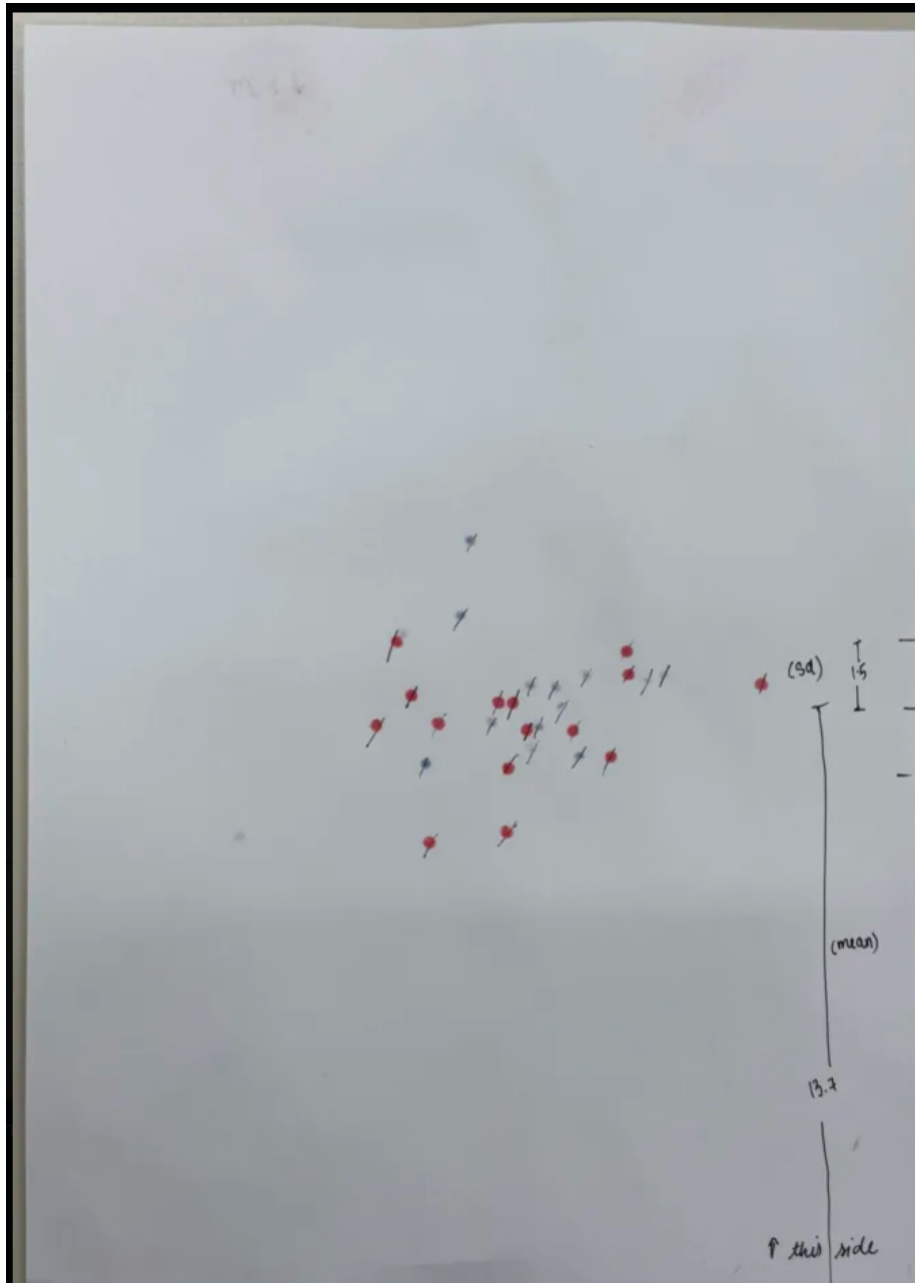


Figure 1: Paper with marble hit markings. (red and black)

```

def uprint(x):
    print(f"{{(x).n:.5f}}") # print number
    print(f"{{(x).s:.5f}}") # print uncertainty

numbers = list(range(1, 17))
mean = statistics.mean(numbers)
std = statistics.pstdev(numbers)
n = len(numbers)
se = std / (n**0.5)

pop_std = statistics.pstdev(numbers)
sam_std = statistics.stdev(numbers)
mean = statistics.mean(numbers)
right_tail = 1 - stats.norm.cdf(1)
left_tail = stats.norm.cdf(1)
two_tail = 1 - (2 * (1 - stats.norm.cdf(1)))

h1 = ufloat(45, 1)
h2 = ufloat(93, 0.5)
d1 = 2 * ((h1 * h2) ** 0.5)
d2 = 2 * ((5 / 7 * h1 * h2) ** 0.5)
print(d1)
print(d2)
listred = [
    14.3,
    14.5,
    15.1,
    12.5,
    13.2,
    13.2,
    13.9,
    13.9,
    12.4,
    10.8,
    10.5,
    13.4,
    13.3,
    14.0,
    15.4,
]
listblack = [
    14.5,
    14.4,
    12.6,

```

```

13.8,
14.2,
14.5,
12.8,
13.3,
13.3,
13.4,
12.5,
16.0,
17.8,
15.5,
13.3,
]
listred = list(map(lambda x: x + 72, listred))
listblack = list(map(lambda x: x + 72, listblack))
print(sum(listred) / len(listred))
print(statistics.stdev(listred))
print(statistics.stdev(listred) / len(listred))
print(sum(listblack) / len(listblack))
print(statistics.stdev(listblack))
print(statistics.stdev(listblack) / len(listblack))

```

– this returns our values as the following, top for list red, bottom for list black. the numbers are in order

$$\mu, \sigma, SE$$

- 85.36 1.3839900907985476 0.09226600605323651
- 86.12666666666668 1.431016954410228 0.0954011302940152
- The code also shows our expected distance for model 1, and expected distance for model [[2]], with

$$h_1 = 45 \pm 1cm, h_2 = 93 \pm 0.5cm$$

- And based off of the code, we find we get an expected

$$d_1 = 129.4 \pm 1.5$$

, for our first model (without angular momentum/energy), and an expected

$$d_2 = 109.3 \pm 1.3$$

for our model [[2]].

- Both of our models have a lower mean and standard error for the distance, indicating there is some [[other]] confounding variable at play here.
- a theoretical drag force is taking place, I suspect it mainly comes from friction from the ramp, and a slight bit of drag from the fall
- but overall our distances being less far matches what you would expect, it would be weird if you were gaining more energy/losing less energy than our calculations.